

***Tower Of Hanoi –
Growth Rate of Output
vs
Time Complexity***

Program:

```
void toh(int n, char src, char dest, char aux)
{
    if (n == 0)
    {
        return ;
    }

    toh(n - 1, src, aux, dest);

    cout << "Move " << n << " from " << src << " to " <<
dest << endl;

    toh(n - 1, aux, dest, src);
}
```

We know , $T(0)$ is the base case, hence constant i.e. $O(1)$, we can use $O(1)$ or C just to represent constant. Not from Time Complexity but like Time Complexity .

Here we have :

Time Complexity : $T(n) = T(n - 1) + T(n - 1) + 1$

Here $O(1)$, for the base case $T(0)$ runs constant time.

But for growth rate of Output:

Step – by – step breakdown

- 1. Move $(n - 1)$ disks from Source $\rightarrow$ Auxiliary
 $\rightarrow$ takes $T(n - 1)$ time***
- 2. Move the largest disk (nth disk)
 $\rightarrow$ takes 1 step $\leftarrow$ THIS is the + 1***
- 3. Move $(n - 1)$ disks from Auxiliary $\rightarrow$ Destination
 $\rightarrow$ takes $T(n - 1)$ time***

Here what we care about:

Growth rate = how fast $T(n)$ increases as $n \rightarrow \infty$.

where , $T(n)$ = number of moves (output size)

Growth Rate of Output:

$$\therefore T(n) = T(n - 1) + T(n - 1) + 1$$

$$T(n) = 2T(n - 1) + 1$$

Note , +1 is negligible in `Growth Rate of Output`

Hence, we can further modify the equation:

$$T(n) = 2T(n - 1) + 1 \approx 2T(n - 1).$$

Now, $T(n - 1) = 2T(n - 2)$

Substituting $T(n - 1) = 2T(n - 2)$ in $T(n)$ we get:

$$\begin{aligned} T(n) &= 2[2T(n - 2)] \\ \Rightarrow 2^2 T(n - 2) \end{aligned}$$

Again, $T(n - 2) = 2T(n - 3)$

Substituting $T(n - 2) = 2T(n - 3)$ in $T(n)$ we get:

$$\begin{aligned} T(n) &= 2^2 T(n - 2) \\ \Rightarrow 2^2 [2T(n - 3)] \\ \Rightarrow 2^3 T(n - 3) \end{aligned}$$

Now, if it runs upto `k` times we get:

$$T(n) = 2^k T(n - k)$$

As, our base case is $T(0)$ not $T(1)$, hence:

$n - k = 0$, therefore, $k = n$, we get:

$$T(n) = 2^n T(n - n)$$

$$\Rightarrow 2^n T(0)$$

Substituting $T(0) = 1$ or C , we get:

$$\Rightarrow 2^n \times 1$$

$$\Rightarrow 2^n$$

$\therefore$ Growth Rate of Output = $O(2^n)$

Alternatively,

$$T(n) = 2T(n - 1) + 1 \approx 2T(n - 1)$$

Here, also we can do, if $T(n) = x^n$

And, x is here is 2,

$$\text{Then, } T(n) = 2^n$$

As $2T(n - 1)$ is homogeneous recurrence,

therefore, we can write directly, $T(n) = x^n$.

Also, in Time Complexity we have seen:

$$T(n) = 2T(n - 1) + 1 \approx O(2^n).$$

That is Growth Rate of Output = Time Complexity here , but its a coincidence here , i. e. not always True.
